Digital Thermometer

Arjun R Iyengar EE26BTECH11202
Arya Sudhirkumar Neel EE26BTECH11203
Tejas L.P EE26BTECH11231

October 2026

Day 1

We spent this day understanding the components, the circuit, and the connections required to be made. To account for the lack of an external ADC, we decided to use a voltage divider circuit with the Arduino's built-in ADC.

Day 2

We started building the circuit using the Arduino's built-in ADC and a voltage divider circuit. However, when we uploaded temporary code to display the temperature, the LCD displayed nothing at all—only the backlight was lit. When we turned the knob of the potentiometer, a few dark lines appeared, but the temperature was never displayed.

We didn't know which component was causing the issue, so we decided to test the LCD first. We made simple connections just to check if the LCD could display "HELLO WORLD" as seen in Figure 1. We uploaded the code and found the same issue again: only the backlight lit up, but nothing was actually displayed. We observed the same few dark lines appearing momentarily when we turned the knob of the potentiometer.

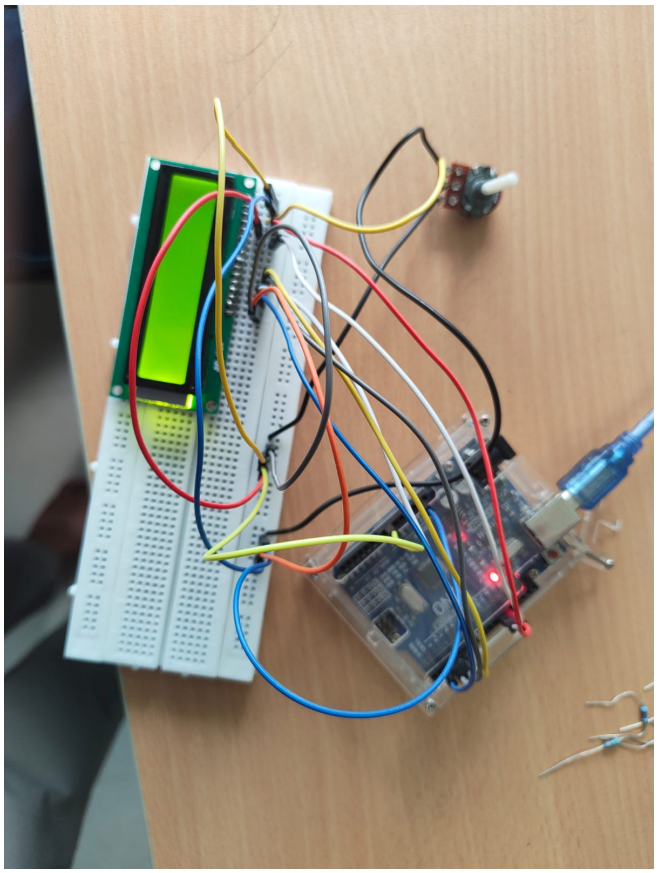


Figure 1: Faulty LCD

We now knew that the potentiometer was working and needed to isolate the error. There could have been various issues, such as faulty breadboard rails; however, that was unlikely since we had used the same breadboard to test previous circuits. The first intuitive step was to replace the LCD display. We borrowed another display from a friend and tested it out. It turned out that the original display was indeed faulty, as the new display worked correctly.

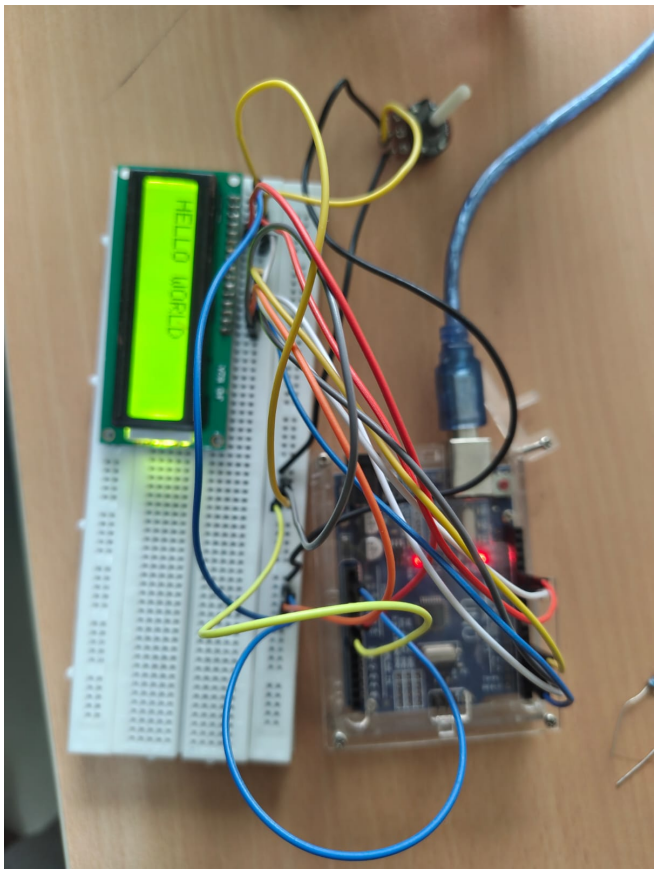


Figure 2: “HELLO WORLD” displayed successfully

Day 3

Now, we could move ahead and verify if the thermometer showed valid readings. We measured the resistance of the reference resistor (119 Ω) and the supply rail ($V_{cc} = 5.08$ V) using a multimeter. We implemented a simple test script without the LSQ model to check baseline readings. However, as expected, there was a deviation of around 5°C. This confirmed that our hardware was functioning correctly and that we needed to apply the LSQ model to minimize measurement error. The test code used was:

```
#include <LiquidCrystal.h>

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

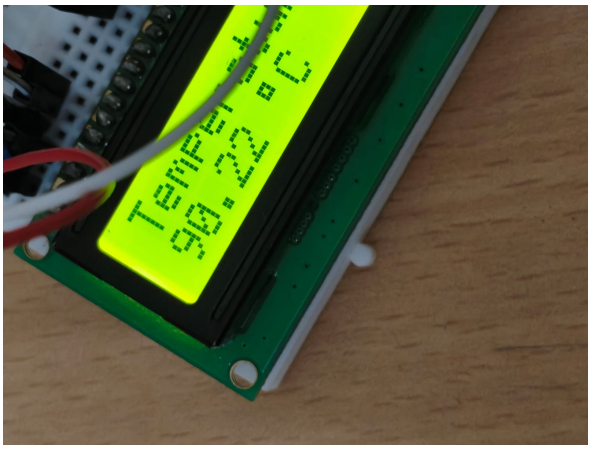
const int sensorPin = A0;
const float Vcc = 5.08;          // Measured supply rail voltage
const float R_fixed = 119.0;     // Measured reference resistor

void setup() {
  lcd.begin(16, 2);
}

void loop() {
  // 1. Oversample 16 readings
  long adcSum = 0;
  for (int i = 0; i < 16; i++) {
    adcSum += analogRead(sensorPin);
    delay(2);
  }
  float avgADC = (float)adcSum / 16.0;
```

```
21
22    // 2. Voltage across PT100
23    float X = avgADC * (Vcc / 1023.0);
24
25    // 3. Calculated PT100 resistance: R_pt = R_fixed * (X / (Vcc - X))
26    float R_pt = R_fixed * (X / (Vcc - X));
27
28    // 4. Standard PT100 conversion (approx 0.3851 Ohms per degree Celsius)
29    float tempC = (R_pt - 100.0) / 0.3851;
30
31    // 5. Display Voltage and Resistance on line 1, Temp on line 2
32    lcd.setCursor(0, 0);
33    lcd.print("V:");
34    lcd.print(X, 2);
35    lcd.print("V R:");
36    lcd.print(R_pt, 1);
37    lcd.print((char)244); // Ohm symbol
38
39    lcd.setCursor(0, 1);
40    lcd.print("Temp: ");
41    lcd.print(tempC, 1);
42    lcd.print(" ");
43    lcd.write(223);
44    lcd.print("C    ");
45
46    delay(400);
47 }
```

The resulting initial outputs were recorded as follows:



(a) Our reading



(b) Actual reference reading

Before proceeding to final calibration, we soldered the circuit onto a perfboard. This required careful layout planning to account for enclosure space and component heights. The soldering process took approximately 5 hours. The assembled board is shown below:

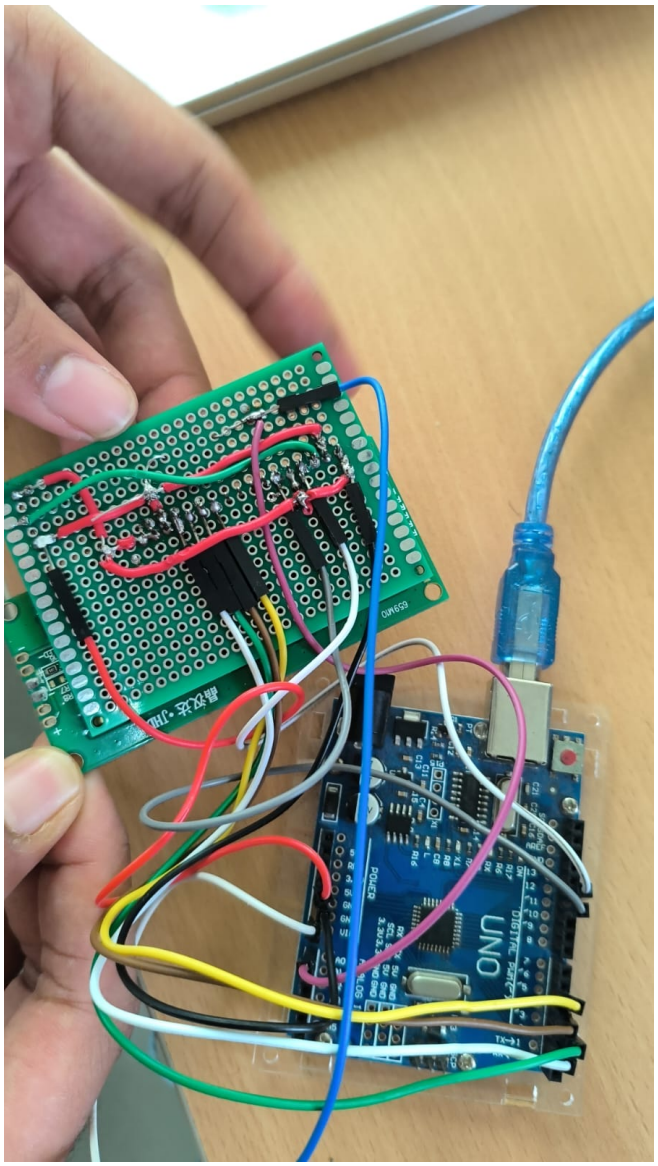


Figure 4: Soldered PCB assembly

Days 4 and 5 - Finding the Right Model

We next attempted to implement the paper's inverse LSQ model with the voltage divider, modifying the coefficients to account for our different reference resistance. However, this approach failed; the room temperature reading alone was offset by approximately 4.5°C.

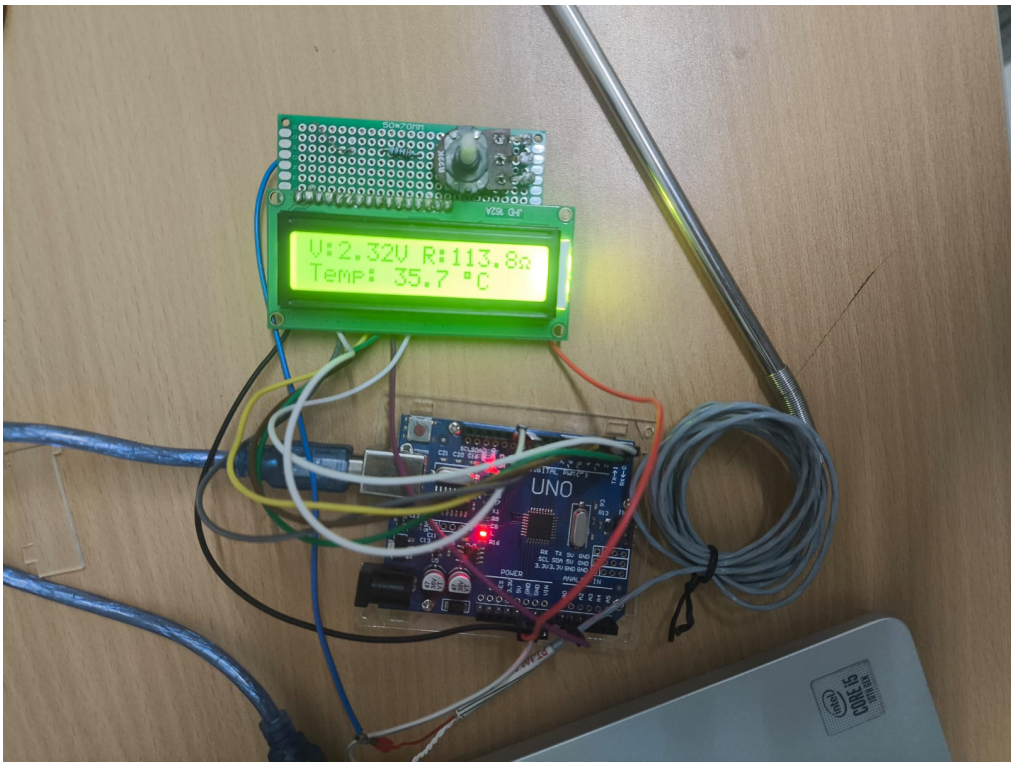


Figure 5: Erroneous inverse model reading at room temperature

This meant we needed to calibrate the setup empirically and derive custom LSQ coefficients. Rather than taking only 3 data points to solve a system of equations, we opted to gather an overdetermined dataset across multiple temperatures and perform a quadratic least squares curve fit using NumPy's `polyfit` function.



Figure 6: Calibration setup in progress

We began calibration by heating water to boiling point and recording data continuously as it cooled, adding cold water to obtain 10°C step intervals down to room temperature and below. Throughout the process, the PT100 probe was fully submerged and stirred continuously to ensure uniform temperature and avoid thermal drift errors. Our initial run yielded 7 samples. Upon evaluating the resulting polynomial fit, we observed:

- Readings between 30°C and 50°C were relatively accurate, with marginal errors of 2–3°C.

- Readings at higher temperatures were significantly off, showing errors up to 7°C.
- Sub-ambient readings were inaccurate due to sparse data coverage at low temperatures.

To address this, we expanded our dataset across the full temperature range, gathering a total of 14 points to improve fit stability. However, a loose wire connection forced us to resolder several joints, requiring a fresh calibration run, which was quite frustrating. The final verified calibration dataset is presented in Table 1:

Table 1: Empirical Calibration Data for PT100 Voltage Divider Circuit ($V_{cc} = 5.05\,\text{V}$)

Sample Point	**Measured Voltage (V) [V]**	**Reference Temp (T) [°C]**
1	2.731183	95.3
2	2.701388	88.8
3	2.671593	77.9
4	2.646764	71.1
5	2.621935	66.3
6	2.602072	61.1
7	2.587175	55.5
8	2.567312	50.5
9	2.542483	44.3
10	2.527585	40.3
11	2.502757	35.9
12	2.487859	32.1
13	2.458065	26.3
14	2.438201	21.8

The Python script used to compute the polynomial coefficients is given below:

```python
import numpy as np

# V and T readings (measured voltage V, reference temperature T)
V = np.array([2.731183, 2.701388, 2.671593, 2.646764, 2.621935, 2.602072,
    2.587175, 2.567312, 2.542483, 2.527585, 2.502757, 2.487859, 2.458065, 2
    .438201])
T = np.array([95.3, 88.8, 77.9, 71.1, 66.3, 61.1, 55.5, 50.5, 44.3, 40.3,
    35.9, 32.1, 26.3, 21.8])

# Least squares polynomial fit of degree 2 (quadratic: T = a*V^2 + b*V + c
    )
coefficients = np.polyfit(V, T, 2)

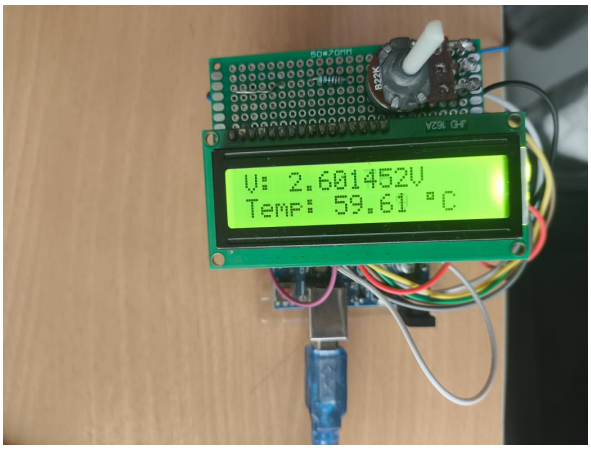
a = coefficients[0]
b = coefficients[1]
c = coefficients[2]

print("Quadratic coefficients:")
print(f"a (V^2): {a:.8f}")
print(f"b (V):   {b:.8f}")
print(f"c (Intercept): {c:.8f}")
print(f"Fitted Equation: T = {a:.4f} * V^2 + {b:.4f} * V + {c:.4f}")
```

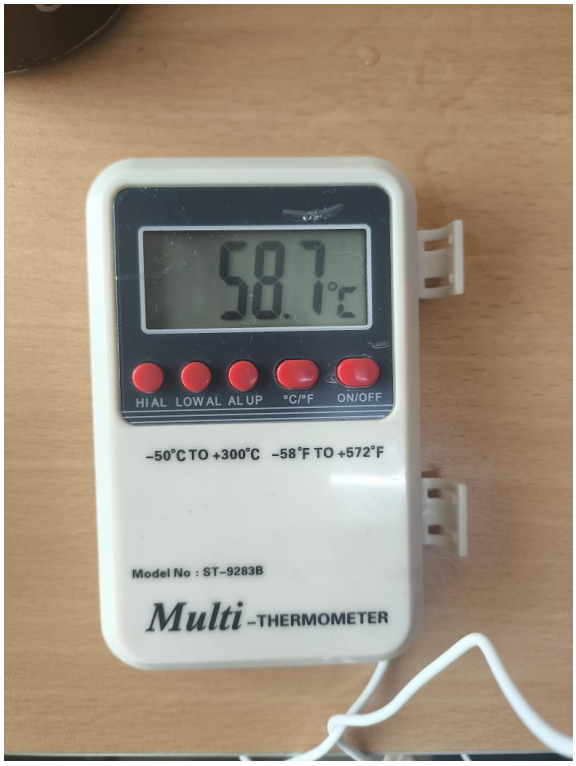
Executing the regression script yielded the following coefficients:

$$a = 170.62478693$$
$$b = -627.58944219$$
$$c = 537.54348189$$

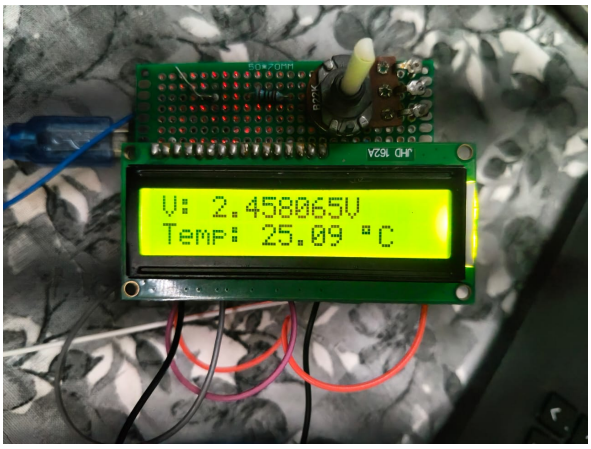
The figures below demonstrate the real-world performance of the calibrated thermometer across different temperature regimes:



(a) Our reading (60°C range)



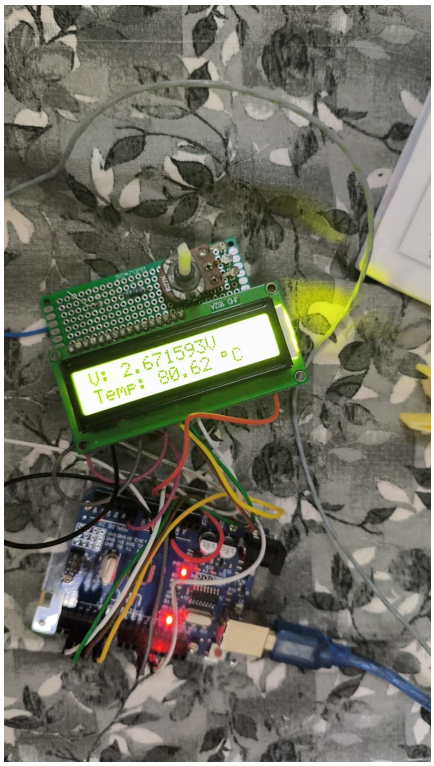
(b) Reference reading



(a) Our reading (25°C range)



(b) Reference reading



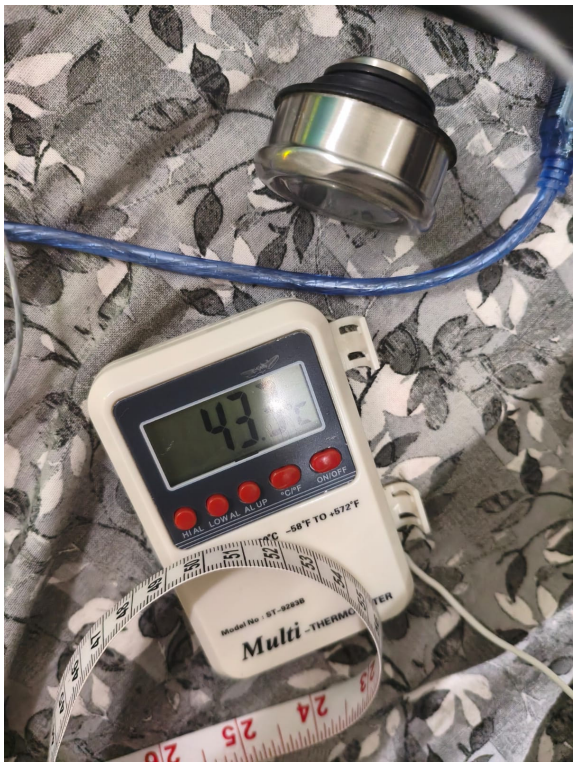
(a) Our reading (80°C range)



(b) Reference reading



(a) Our reading (40°C range)



(b) Reference reading



(a) Our reading (30°C range)



(b) Reference reading

The calibration successfully reduced measurement deviations to within 0.4–1.5°C across the entire operating range.

Days 6 and 7 - CAD Modeling

The final stage involved CAD design for the physical enclosure. We designed a custom two-part casing in FreeCAD to house the Arduino Uno, soldered board, LCD screen, and potentiometer.

We began by sketching the baseline perimeter and applying a pad extrusion:

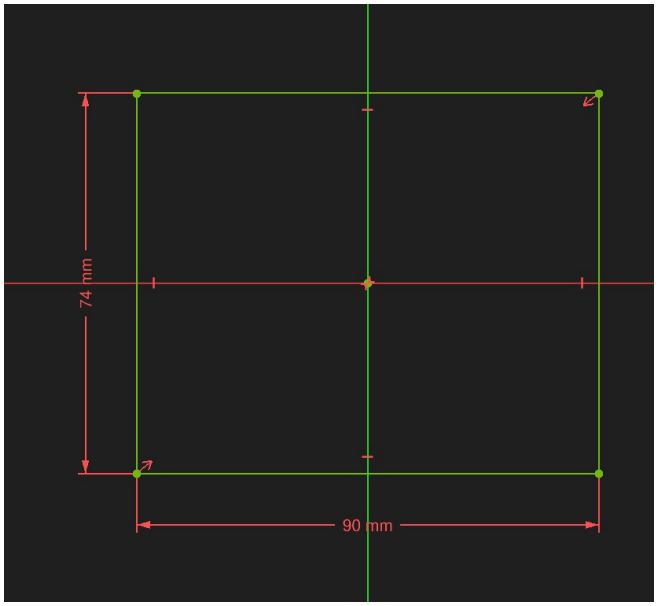


Figure 12: Initial Sketch

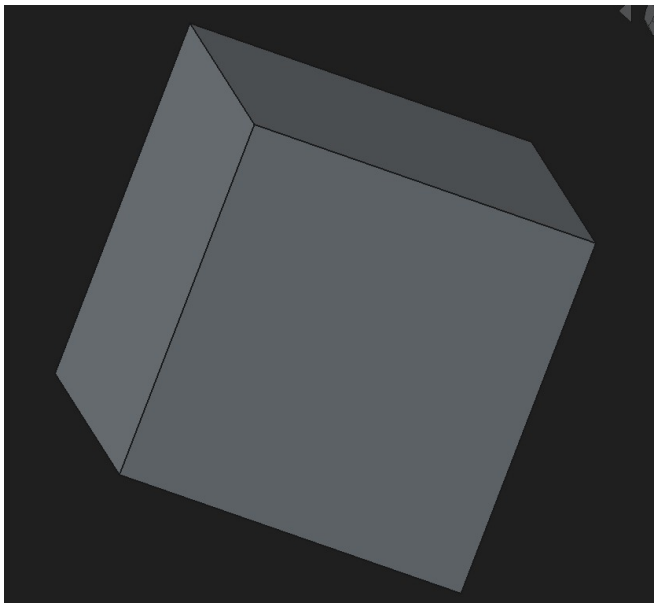


Figure 13: Pad Extrusion

Next, we hollowed out the interior body volume and designed mounting lips for the top lid:

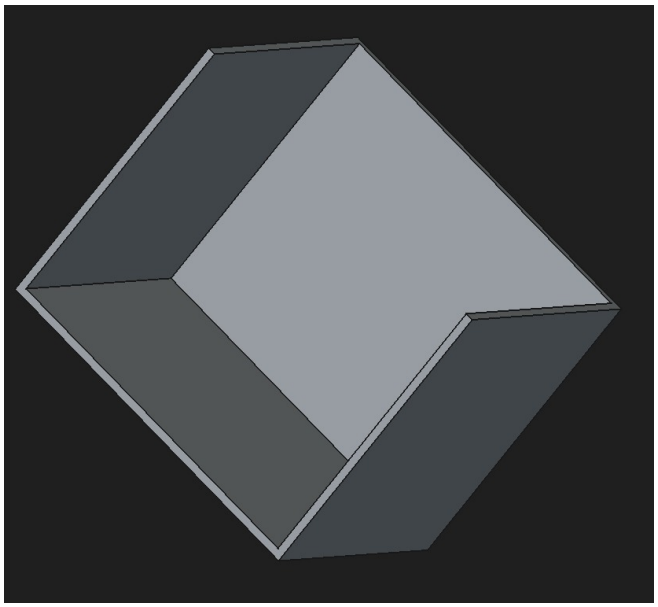


Figure 14: Internal cavity hollow

We then added standoffs and screw holes for securing the Arduino board:

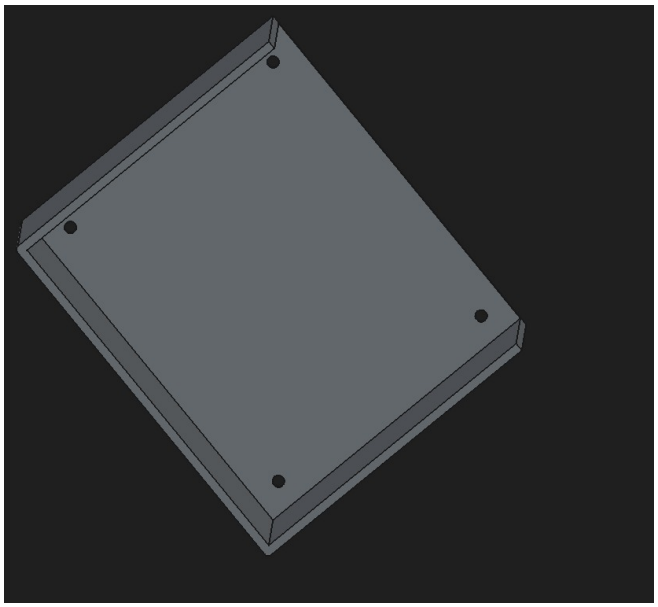


Figure 15: Arduino mounting points

Threaded mounting posts were added to fasten the top cover to the main body:

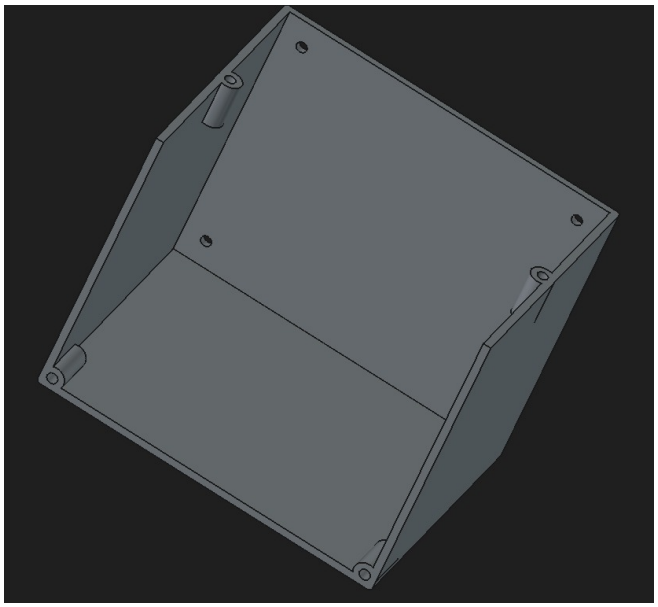


Figure 16: Cover mounting standoffs

Internal support rails and PCB mounting pegs were added next:

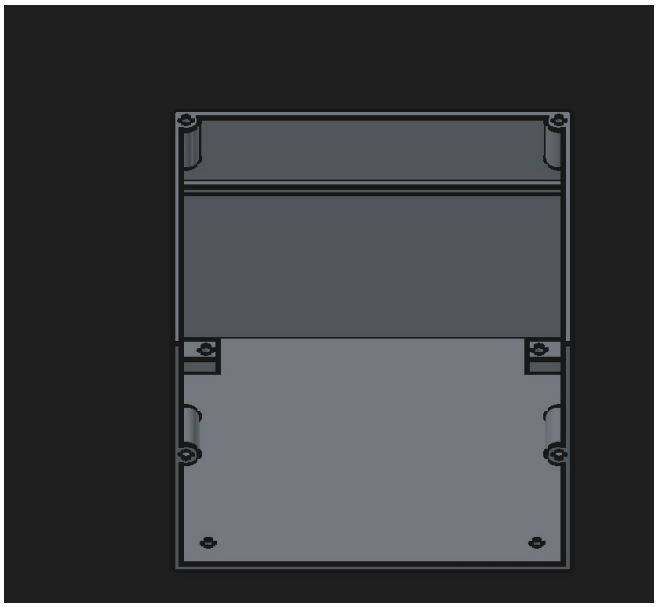


Figure 17: PCB support structure

A cable spool feature was incorporated to manage excess PT100 lead wire length:

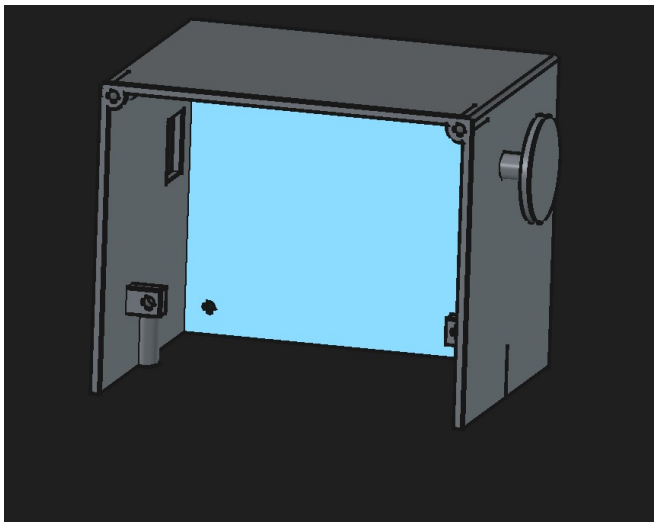


Figure 18: Integrated cable spool

We then added an external storage holder clip for securing the PT100 probe:

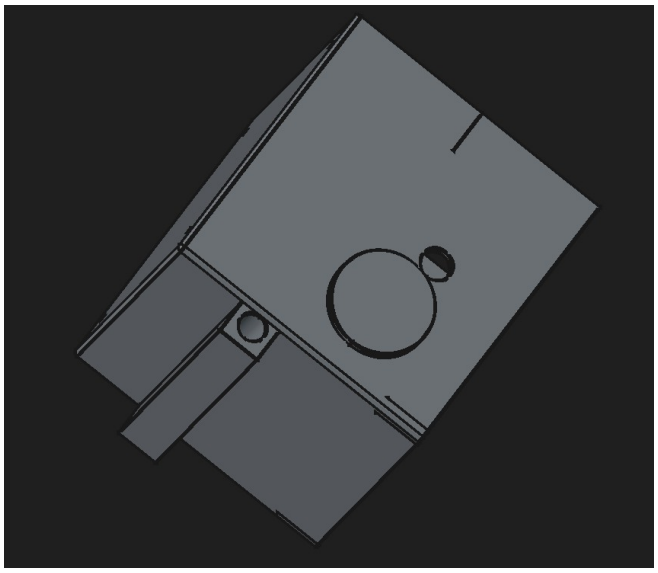


Figure 19: PT100 probe holster

Finally, cutouts for the 16x2 LCD display screen and contrast adjustment potentiometer knob were added to the lid, completing the 3D enclosure model:

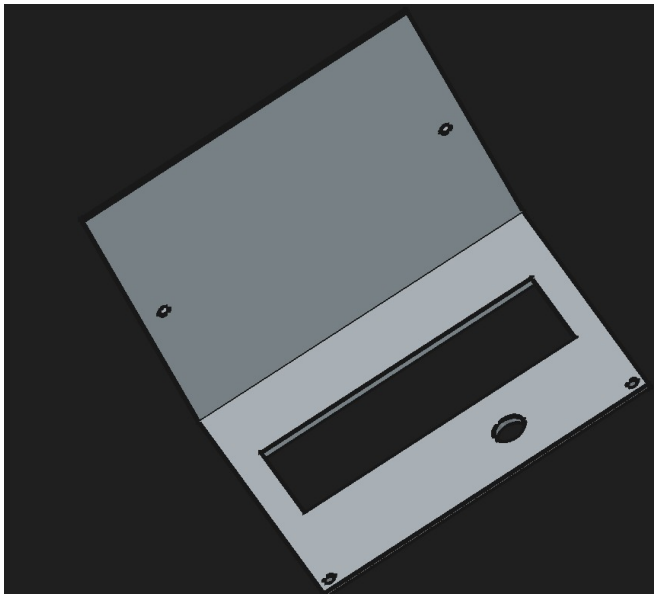


Figure 20: Final assembled CAD model

Learnings

- Gained practical troubleshooting experience by systematically isolating individual hardware components.
- Learned the necessity of custom Least Squares (LSQ) regression when working with non-ideal component tolerances and differing supply voltages ($V_{cc} = 5.08\,\mathrm{V}$) Relying on theoretical or paper models introduced systematic offsets, whereas collecting an overdetermined empirical calibration dataset (V, T) directly mapped out lead wire resistance and non-linear PT100 behavior.
- Learned how mechanical constraints dictate electrical component layout. Pivoting from a sliding-lid mechanism to a screwed-down enclosure demonstrated the importance of eval-

uating 3D-printing tolerances, structural rigidity, and ease of assembly during early CAD prototyping.

- Improved project management and agile problem-solving under a hard deadline. Iterative brainstorming allowed us to quickly adapt, re-calibrate after wire re-soldering, and successfully optimize model precision to within 0.4–1.5°C.

Future Work

We wish to explore machine learning data augmentation and regularization techniques (e.g., Gaussian noise injection or Mixup interpolation) to train higher-capacity regression models if deployed across non-linear sensor networks.